

Deployment Guide

Run the trained supervisory network as a shadow/advisory worker above PLC/PID/SIS

The deployment half of the pipeline: take the JNeopallium network the trainer produced — together with its ready-made production context — wire it to your OPC UA, MQTT, and Kafka feeds, and run it as a continuous shadow-to-advisory supervisory worker that never touches deterministic control.

Document	Production Deployment Guide
Product	Jneopallium Industrial Loop Guardian (FMI Skid Demo)
Consumes	The deployable network from the Training Guide
Safety mode	ADVISORY (supervisory, recommend-only)
Platforms	Windows (PowerShell) and Linux/macOS (bash)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1. What you will achieve
2. Prerequisites — the trained package and runtime
3. The launch architecture
4. The deployable package (what training produced)
5. Point the worker at the generated neuron-net structure
6. Package the deployment JAR
7. Runtime configuration: production-context.json
8. Configure event sources and streaming input
9. Wire the advisory output and launch the worker
10. The control and safety boundary in production
11. Promote: shadow → advisory → bounded autonomous
12. Monitoring in production
13. Safety, rollback and packaging
14. Troubleshooting
15. Appendix — deployment cheat-sheet

1 What you will achieve

By the end of this guide you will have taken the **deployable network produced by the Training Guide** — a directory of JNeopallium layer/neuron configuration plus a ready-made `production-context.json` — and run it as a continuous **supervisory advisory worker** that diagnoses pump wear, oscillation, sensor drift, and energy deterioration, and recommends — but never autonomously controls.

Safety ceiling for the whole guide

Everything below operates in ADVISORY mode, above PLC/PID/SIS. The worker may recommend maintenance, bounded setpoint trims, and energy actions; it must not actuate. OPC UA actuator writes stay blocked until an offline replay, a shadow pilot, an advisory subscription, and a site safety case have all passed.

2 Prerequisites — the trained package and runtime

- **A trained network** from the Training Guide — the artifact directory containing `model-descriptor.json`, `layer-0.json` ... `result-layer.json`, `trained-industrial-loop-guardian-model.json`, and `production-context.json`.
- **Java 17+** and the built worker (`worker-jar-with-dependencies.jar`).
- **Your plant telemetry** — OPC UA process tags, MQTT IIoT telemetry, and optionally a Kafka shadow stream and CMMS / energy-meter context.
- **An output destination** — a JSONL sink for shadow mode, or Kafka / SIEM / CMMS / webhook for operator-visible advisory.

Training first

If you do not yet have a trained network, run the companion Training Guide. Deployment consumes its output directory directly, including the bundled `production-context.json`.

3 The launch architecture

A production deployment uses the worker's real entry point with four arguments:

```
java com.rakovpublic.jneuropallium.worker.application.Entry \  
  <mode> <model-jar-url> <context-class> <context-json-or-path>
```

Argument	Meaning	Example
mode	Deployment topology	local, http, grpc-client, grpc-master
model-jar-url	URL of your deployment JAR	file:///opt/jneopallium/industrial-loop-guardian.jar
context-class	An IContext implementation	...runtime.DemoJsonContext
context-json-or-path	Inline JSON or a path to a JSON config file	.../production-context.json

Entry deserialises the context with the context class, loads the model JAR into an isolated class loader, then maps `local`, `http`, or `grpc` to `LocalApplication`, `HttpClusterApplication`, or the gRPC server path — the same topologies as the architecture.

4 The deployable package (what training produced)

The trainer emits the **actual JNeopallium network**, not an abstract model. The five layer files plus the descriptor are what the worker boots from:

Artifact	Role at deploy time
<code>model-descriptor.json</code>	Whole-network map: 5 layers, 17 neurons, frequency map, runtime classes
<code>layer-0.json</code>	Plant + supervisory-context input boundary
<code>layer-1-fast-telemetry.json</code>	Fast telemetry validation and loop state (7 neurons)
<code>layer-2-maintenance-energy.json</code>	The four trained diagnostic finding heads
<code>layer-3-advisory-planning.json</code>	Economic advisory planning + the fixed safety gate (4 neurons)
<code>result-layer.json</code>	Industrial advisory JSONL output (2 neurons)
<code>production-context.json</code>	A ready-to-run advisory-worker context

Total network: **5 layers, 17 real neurons, 156 trainable weights, 4 biases**. Every neuron names a concrete class from the worker's industrial package — deployment points the worker at this directory, it does not re-implement anything.

5 Point the worker at the generated neuron-net structure

The neuron-net structure is **generated by training**, not hand-written. `configuration.input.layermeta` (set in `production-context.json` to `model/industrial-loop-guardian/`) points at the directory holding the five layer files, and the worker boots the network directly. Each file is a real layer of real neurons:

Layer file	Real neuron classes / role
<code>layer-1-fast-telemetry.json</code>	<code>SensorNeuron</code> , <code>MeasurementValidatorNeuron</code> , <code>OscillationMonitorNeuron</code> , ...
<code>layer-2-maintenance-energy.json</code>	<code>DegradationModelNeuron</code> , <code>EnergyAccountingNeuron</code> — the four heads
<code>layer-3-advisory-planning.json</code>	<code>MaintenanceSchedulingNeuron</code> , <code>SetpointOptimiserNeuron</code> , <code>SafetyGateNeuron</code>
<code>result-layer.json</code>	Advisory JSONL output neurons

The diagnostic heads embed everything the model learned, ready to run — the trained weights, each head's allowed `featureGate`, its `logicalNeuronRole`, and its owned `Reasoning` — so a production reviewer can confirm the logic lives inside the model, not in glue code.

No translation step

Because the structure is generated with the trained heads already inside, what you deploy is exactly what was trained and tested: the same weights, the same feature gates, the same response bands, the same fixed safety gate.

6 Package the deployment JAR

The neuron, processor, and signal classes the structure references **already ship in the worker's industrial module**, so you do not re-implement them. Package these for deployment:

Component	Contents
Typed signals	MeasurementSignal, AlarmSignal, DegradationSignal, EfficiencySignal, MaintenanceWindowSignal, SetpointSignal, InterlockSignal
Neurons	SensorNeuron, MeasurementValidatorNeuron, OscillationMonitorNeuron, DegradationModelNeuron, MaintenanceSchedulingNeuron, EnergyAccountingNeuron, SetpointOptimiserNeuron, SafetyGateNeuron
Processors	validation, oscillation/stiction, degradation scheduling, maintenance-window scheduling, efficiency optimisation, safety gate, dispatch
Sources	OPC UA measurements/alarms, MQTT telemetry, Kafka shadow, CMMS, energy-meter context
Sinks	JSONL audit sink, Kafka/SIEM/webhook adapter; OPC UA command sink only with a safety case

Fail-fast class check

`production-context.json` sets `configuration.neuronnet.classes` to the full runtime class list. At startup `LocalApplication` checks every class is present in the JAR and fails fast if the package is incomplete.

7 Runtime configuration: production-context.json

The trainer generates a ready-to-run context. It uses the standard shape `{"properties": { ... }}` and already encodes the advisory-loop settings. The critical keys:

Key	Production value
<code>configuration.input.layermeta</code>	<code>model/industrial-loop-guardian/</code> (the generated layer directory)
<code>configuration.isteacherstudying</code>	<code>false</code>
<code>configuration.discriminatorsAmount</code>	<code>0</code>
<code>configuration.infiniteRun</code>	<code>true</code>

<code>configuration.runoncein</code>	1 ms (one advisory tick per millisecond)
<code>configuration.processing.frequency.map</code>	fast tags every loop; degradation/energy every 10; maintenance every 60
<code>configuration.outputAggregator</code>	JsonlResultAggregator (swap for your SIEM/Kafka sink)
<code>industrial.autonomousAction</code>	false
<code>industrial.opcua.role</code>	bounded-local-actuator-path
<code>industrial.mqtt.role</code>	telemetry-and-advisory-only
<code>industrial.neuronOwnedLogic</code>	diagnosis, economic-basis, safety-envelope, bounded-recommendation

The continuous advisory loop

Together, `isteacherstudying=false`, `discriminatorsAmount=0`, and `infiniteRun=true` put the worker into its continuous advisory loop. `runoncein=1` paces it to one advisory tick per millisecond, before the per-signal frequency map fans fast and slow signals out at their own cadences.

8 Configure event sources and streaming input

Local Entry replay uses the bundled `IndustrialLoopGuardianReplayInput` (started by the `IndustrialLoopGuardianEntryLauncher`). For production, plug in a site-specific `IInitInput`, or one of the shipped bridge inputs — **OPC UA, MQTT, FMI, and PLC4X** measurement/event inputs — that convert your records into typed industrial signals while preserving event time. Recommended Kafka topic groups:

Topic group	Maps to signal
<code>plant.telemetry.measurements</code>	<code>MeasurementSignal</code>
<code>plant.telemetry.alarms</code>	<code>AlarmSignal</code>
<code>plant.maintenance.events</code>	<code>DegradationSignal</code> / maintenance context
<code>plant.energy.meters</code>	<code>EfficiencySignal</code>
<code>plant.cmms.workorders</code>	<code>MaintenanceWindowSignal</code> / maintenance history
<code>plant.condition.waveforms</code>	<code>MachineWaveformSignal</code> (acoustic / vibration streams)

Keep the streaming cadence aligned with `configuration.processing.frequency.map`: fast-loop tags every loop; maintenance and energy findings every 10–60 loops; machine-health features and advisories on their own slower cadence. Because correlation is by event time, delayed or out-of-order telemetry still lands in the right place.

Machine-health condition-monitoring input

Acoustic and vibration waveforms arrive as `MachineWaveformSignal`; the Java machine-health runtime extracts features, scores operating regime and domain shift, forms fault hypotheses, and emits an advisory — all read-only. Feed it from the same OPC UA / MQTT / FMI / PLC4X bridges, or a dedicated high-rate sensor stream.

9 Wire the advisory output and launch the worker

`configuration.outputAggregator` names an `IOutputAggregator`, called every cycle with the advisories the net produced. The bundled `JsonlResultAggregator` writes advisory JSONL — ideal for the shadow-mode audit sink. Each advisory carries its full reasoning and economic basis:

```
{
  "asset": "P-101",
  "finding": "pump wear and cavitation risk",
  "confidence": 0.73,
  "evidence": { "neuron": "PumpHealthAndEfficiencyNeuron", "maxVibrationRms": 4.3 },
  "recommendation": "SCHEDULE_PUMP_INSPECTION",
  "recommendedAction": "Inspect P-101 impeller and suction strainer.",
  "urgencyHours": 48,
  "economicBasis": { "neuron": "EconomicBasisNeuron",
    "estimatedAvoidedShutdownValueUsd": 20000,
    "safetyEnvelopeSatisfied": true },
  "controlBoundary": { "plcPidSis": "deterministic control and hard safety remain authoritative",
    "jneopallium": "supervisory diagnosis and bounded recommendations"
  },
  "autonomousAction": false
}
```

The machine-health subsystem emits a parallel `MachineHealthAdvisorySignal` with its own calibrated, uncertainty-aware shape — also advisory-only:

```
{
  "asset": "P-101",
  "healthScore": 0.62,
  "anomalyProbability": 0.71,
  "faultProbabilities": { "bearingDamage": 0.66, "cavitation": 0.21,
    "imbalance": 0.08, "sensorFault": 0.05 },
  "unknownAnomalyProbability": 0.12,
  "domainShiftScore": 0.18,
  "uncertainty": 0.24,
  "evidence": { "neuron": "MachineHealthCorrelationNeuron",
    "acousticRms": 0.41, "vibrationEnvelopeEnergy": 0.58 },
  "autonomousAction": false
}
```

For production, forward this JSONL to your SIEM, a Kafka topic, a CMMS, or a webhook. Then launch the worker (use `;` as the classpath separator on Windows, `:` on Linux/macOS):

```
java -cp worker/target/worker-jar-with-dependencies.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local \
  file:///opt/jneopallium/industrial-loop-guardian.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  /opt/jneopallium/model/industrial-loop-guardian/production-context.json
```

10 The control and safety boundary in production

The boundary the demo enforces is the boundary you keep in production. It is what makes the product insurable and easy to validate:

- **PLC / PID / SIS** own deterministic millisecond control and hard safety. Always authoritative.
- **OPC UA** is the only bounded actuator path, and only when a safety case allows writes. Three command nodes; every write passes the priority chain below.
- **MQTT** is telemetry and advisory only — the bridge structurally rejects AUTONOMOUS.
- **Machine-health** is read-only: the `MachineHealthAdvisorySignal` never writes an actuator command; the interlocks, safety gate, and operator override stay authoritative over it too.

```
hard interlock -> local fail-safe -> operator override -> safety mode
-> validation/quality -> clamp -> ramp limit -> diff suppression
-> OPC UA write -> audit
```

Fail-safe defaults on loss: cooling valve 100% open, pump speed 30%, heater power 0%. On OPC UA loss the skid applies these locally; on MQTT loss fast-loop control availability stays at 1.0.

11 Promote: shadow → advisory → bounded autonomous

Promote gradually. Each step earns the next; OPC UA writes stay blocked until the whole sequence has passed.

1. **Offline replay.** Analyse exported historian / OPC UA / MQTT data without connecting to production. Deliver discovered anomalies, an event timeline, estimated financial impact, and a false-positive review.
2. **Shadow pilot.** Read-only connection to the plant, no actuator writes. Compare advisories against actual later outcomes and a simple-algorithm baseline.
3. **Production advisory subscription.** Operator-visible advisories with economic basis, once false-positive budgets and latency targets are met.
4. **Bounded autonomous optimisation.** Only after substantial shadow evidence and a site safety case — narrowly bounded setpoint recommendations or commands that never replace PLC interlocks or a certified SIS.

12 Monitoring in production

- Measurements processed per source per minute; dropped, delayed, out-of-order events.
- Advisory count per asset; false-positive review rate; confirmed true-positive rate.
- Detection lead time (how early before the eventual failure or alarm).
- Energy and downtime estimates versus a simple-algorithm baseline.
- Model version and `trainingChecksum` used for every advisory; interlock and override events.

Every advisory preserves its lineage: asset, contributing signals, the owning neuron, the economic basis, the safety-envelope result, and the control-boundary statement.

13 Safety, rollback and packaging

Packaging

- The generated artifact directory (layer files, model, descriptor, production-context.json).
- Git commit SHA, training command, the trainingChecksum, generated artifact checksums, the advisory-deployment approval record, and a rollback package identifier.

Safety and rollback rules

- Hard safety gates are fixed configuration in layer-3-advisory-planning.json, never learned model weights; interlocks and operator overrides remain authoritative.
- Keep the previous network package available for immediate rollback.
- Roll back if false positives exceed budget, source coverage drops, or the simple-algorithm baseline is no longer beaten.

Honest limitation

The checked-in reference network is trained on a synthetic, deterministic skid corpus. It is excellent for repeatable pipeline evidence and safety-boundary regression, but it is simulation/HIL evidence — not a certified safety controller. It does not replace PLC/SIS validation, hazard analysis, management of change, or site acceptance testing. Real accuracy requires external historian, CMMS, and energy-meter validation.

14 Troubleshooting

Symptom	Likely cause and fix
Cannot find class ... in provided jar	A class in configuration.neuronnet.classes is missing — rebuild the deployment JAR
Worker starts but emits nothing	Check the event source clazz and that readSignals() returns typed signals
Structure not loaded	Confirm configuration.input.layermeta points at the generated layer directory
Worker exits immediately	For a live supervisor set infiniteRun=true; maxRun is for bounded replay
An advisory tries to actuate	Verify industrial.autonomousAction=false and the response layer is ADVISORY
Stale weights at runtime	You deployed an old layer directory; redeploy the latest trained package

15 Appendix — deployment cheat-sheet

Launch a production advisory worker

```
java -cp worker/target/worker-jar-with-dependencies.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///opt/jneuropallium/industrial-loop-guardian.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
```

```
/opt/jneopallium/model/industrial-loop-guardian/production-context.json
```

The advisory-worker context essentials

```
configuration.input.layermeta    = model/industrial-loop-guardian/  
configuration.isteacherstudying = false  
configuration.discriminatorsAmount = 0  
configuration.infiniteRun       = true  
configuration.runoncein         = 1  
industrial.autonomousAction      = false
```

Jneopallium Industrial Loop Guardian · Deployment Guide. Consumes the network produced by the Training Guide. Supervisory above PLC/PID/SIS. Safety mode: ADVISORY. OPC UA writes require a separate site safety case. License: BSD 3-Clause.